

Mobile Devices and Applications

UNIT 4 – APP LIFECYCLE AND UI

Summary



App Components



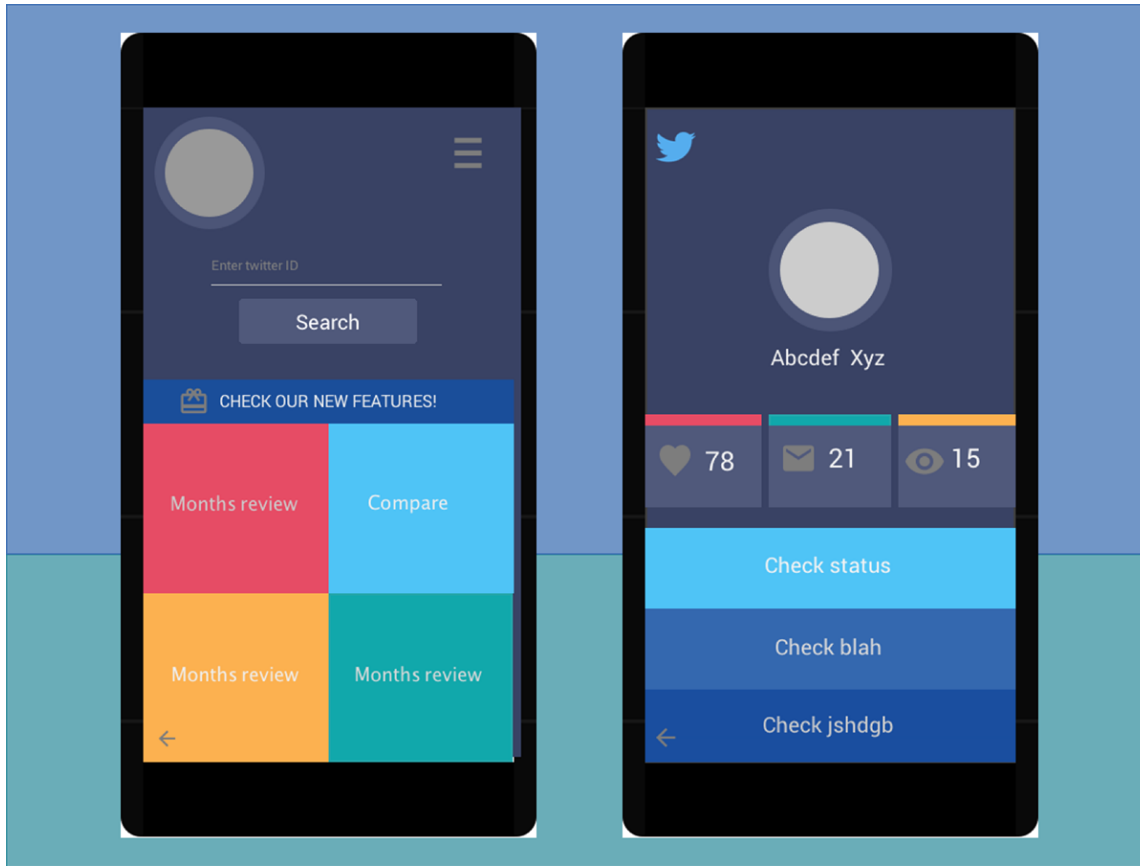
XML Layouts



Procedural Interface



Containers



User Interface

Declared Interfaces

XML files

Definition

- Graphical elements (**res/layout**)
- Menus (**res/menu**)

They can be reused, included

Make use of resources

- Constant values (**res/values**)
 - strings
 - dimens
 - colors

Binding activity class by a user interface

- **setContentView()**

XML Files (layout)

Includes important visual components displayed by an activity

Each included element is represented by an XML tag

The name of an element is the same with the name of the class used to create objects inside activities

The properties of an element are the attributes of an object of the same type

Properties

android:*property*

- for each individual element

android:layout_*property*

- found in LayoutParams class
- send to the parent for positioning an element

mandatory:

- **android:layout_width** and **android:layout_height**
 - Values:
 - **wrap_content** or **match_parent** or **constant values**

XML Resources

@package:resourceType/resourceName

In the current project

- *@resourceType/resourceName*
- Example:
 - **@dimen/dim_font**
 - **@color/culoare_fundal**

Predefined resources

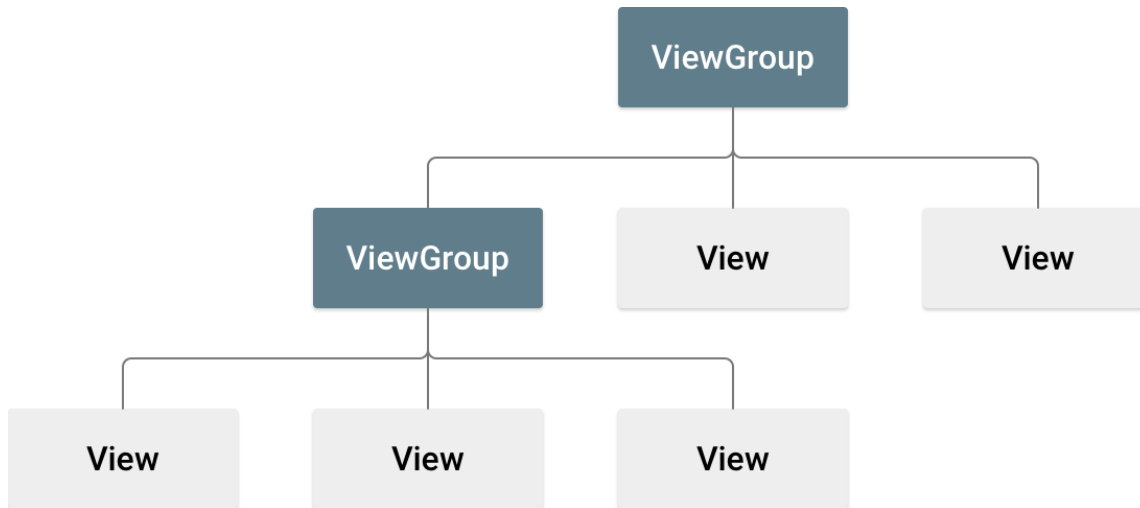
- *@android:resourceType/resourceName*
- Example
 - **@android:color/background_light**
 - **@android:string/dialog_alert_title**

Layouts

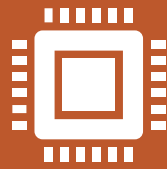
A layout defines the structure for a user interface in your app.

All elements in the layout are built using a hierarchy of View and ViewGroup objects.

The view objects are called widgets.



Layouts declaration



Declare UI elements in XML using the Android XML syntax found in Android namespace;



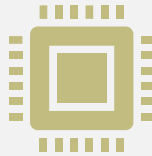
Instantiate layout elements at runtime;

Using elements inside code



a method inside the
Activity class:

```
View findViewById(int  
resourceIdentifier)
```



Example

```
TextView tv = (TextView)  
findViewById(R.id.textVie  
wTitlul);  
tv.setText("text stabilit din  
cod");
```



The call should be made after the
`setContentView()`

px (pixels) - actual screen pixels. Use is not recommended since different displays will have different pixels per inch. Use dp instead of this unit.

dp (density-independent pixels) - an abstract unit of measurement based on the physical density of the device display relative to a 160dpi display baseline, one pixel for a 160dpi

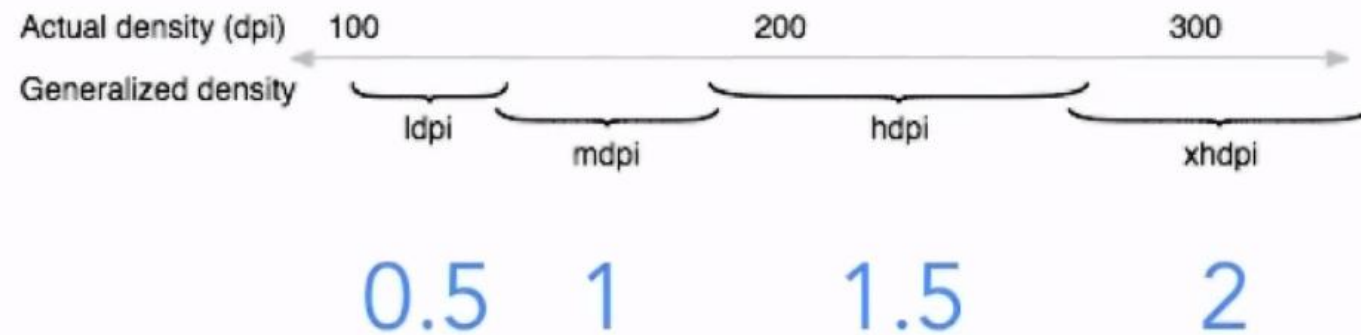
mm, in – refers to the actual screen size

pt (points; 1/72 inch), fonts

sp (scale-independent pixels) - scale-independent pixels. Similar to dp but scaled based on the user's font preference.

Screen size

Screen size conversions



$$dp = px * 160/dpi$$

$$px = dp * dpi/160$$

Normal density

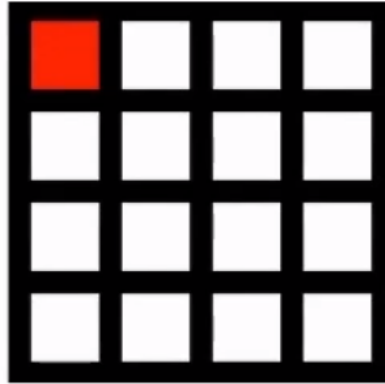


1px

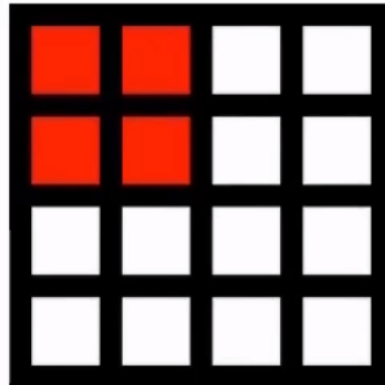


1dp

Double density



1px



2px

Screen density

Scaling images - the 3:4:6:8:12:16 ratio rule

36x36 (0.75x) for
low-density (ldpi)

**48x48 (1.0x
baseline) for
medium-density
(mdpi)**

72x72 (1.5x) for
high-density (hdpi)

96x96 (2.0x) for
extra-high-density
(xhdpi)

144x144 (3.0x) for
extra-extra-high-
density (xxhdpi)

192x192 (4.0x) for
extra-extra-extra-
high-density
(xxxhdpi)

Gesture sensitivity

BASELINE SCREEN (160 DPI)

If the app requires 16 pixels to detect a fling action than:

$16/160 \text{ dpi} = 1 / 10^{\text{th}}$ of an inch

approx. 2.5 mm

HIGH DENSITY SCREEN (240 DPI)

If the app requires 16 pixels to detect a fling action than:

$16/240 \text{ dpi} = 1 / 15^{\text{th}}$ of an inch

approx. 1.7 mm

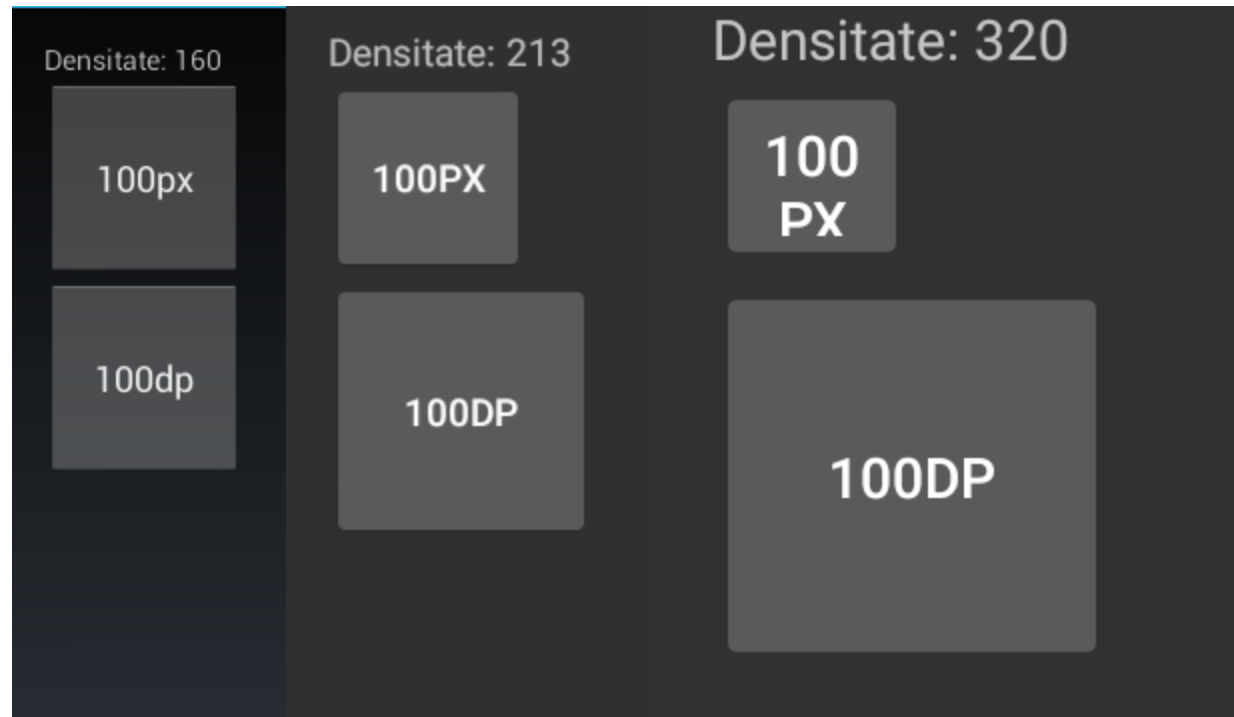


Screen dimensions

Code example:

```
DisplayMetrics dm = getResources().getDisplayMetrics();  
//to pixel conversion  
int px = (int) TypedValue.applyDimension( TypedValue.COMPLEX_UNIT_DIP, dp, dm);  
//or  
int px = Math.round(dm.density * dp);
```

Dimensions



3.5", 240x320

7", 800x1280

4.7", 768x1280

Properties

android:id – component id

Associating an id:
"@+id/identifier"

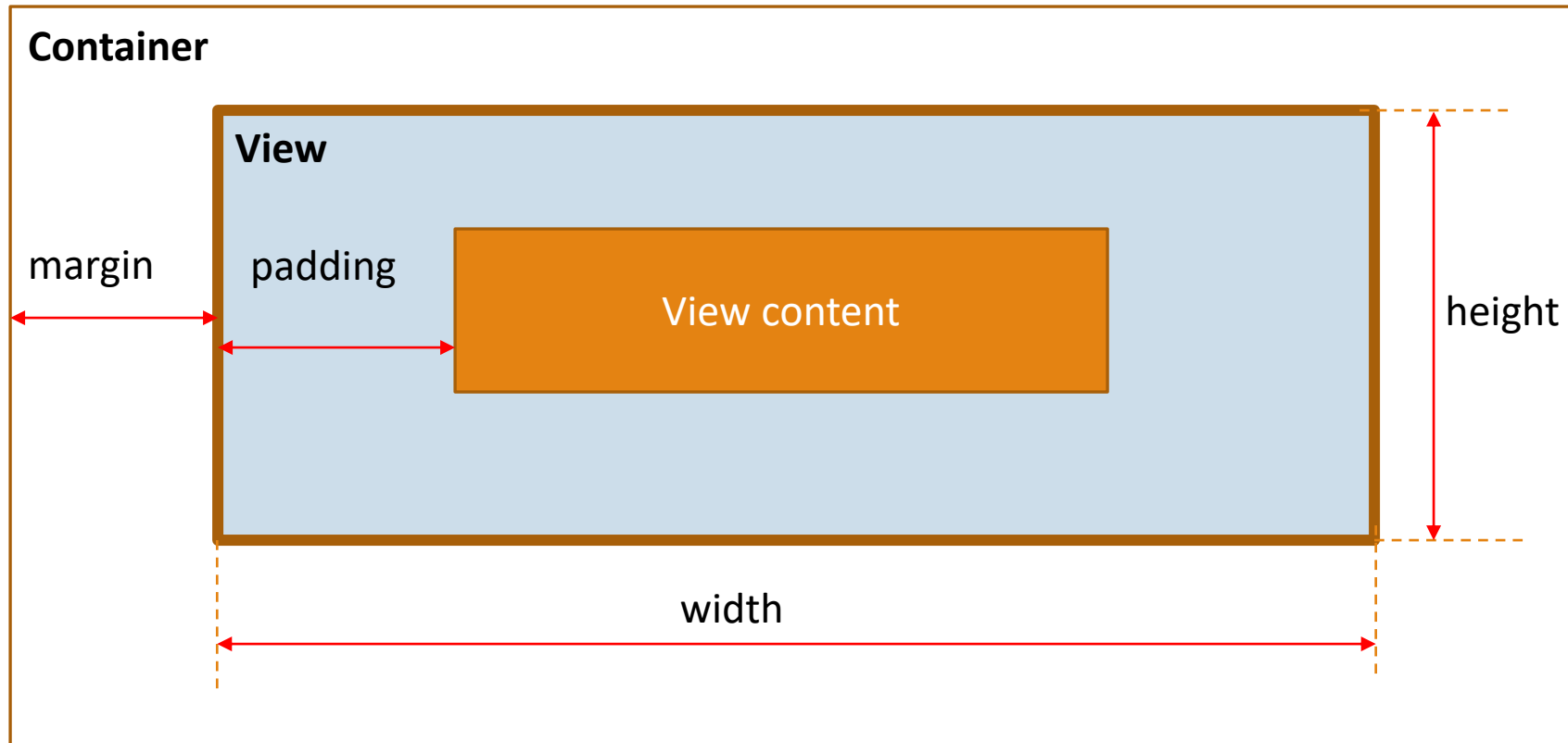
Using an identifier:

- XML: @id/identifier
- Java: R.id.identifier

Properties

Property	Attribute	Example
Text	android:text	"Text în clar" @string/text_control
Distance inside the view itself	android:padding android:paddingTop Left Right Bottom	10dp
Background colour	android:background	#FF00FF @drawable/imag
Alignment	android:gravity	center, left
Distance outside the view	android:layout_margin, android:layout_marginTop Left Right Bottom	30dp
Visibility	android:visibility	visible, invisible, gone

Properties



PROCEDURAL INTERFACE

A thin vertical line is positioned to the right of the text, extending from the top of the word 'PROCEDURAL' down to the bottom of the word 'INTERFACE'.

PROCEDURAL

Create runtime controls

Constructors

- The current context

Initializing properties

Adding the controls to a container

- `addView()` method

Displaying the container

- `setContentView()`

Common properties

Dimensions:

- getWidth()
- getHeight()

Appearance:

- setBackgroundColor(int culoare)
- setBackground(Drawable)
- setBackgroundResource(int idResursa)

Common properties

Spacing (distance between the margin of the control and the content itself):

- `setPadding()`
- `getPaddingLeft()`
- `getPaddingTop()`
- `getPaddingRight()`
- `getPaddingBottom()`

Active (get or set the enabled property)

- `setEnabled(boolean)`
- `isEnabled()`

Focus (get or set the focusable property)

- `setFocusable(boolean)`
- `isFocusable()`

Common properties

Container parameters

- `setLayoutParams(LayoutParams)`
- `LayoutParams getLayoutParams()`

Visibility

- `setVisibility(int)` – `VISIBLE`, `INVISIBLE`, `GONE`

Common properties

Position (pixels):

- absolute
 - `setX()/getX()`
 - `setY()/getY()`
- parent relative
 - `setLeft()/getLeft()`
 - `setTop()/getTop()`
 - `setRight()/getRight()`
 - `setBottom()/getBottom()`

Common properties

Events management:

- `setOnClickListener()`
- `setOnLongClickListener()`
- `setOnDragListener()`
- `setOnKeyListener()`
- `setOnTouchListener()`

Procedural interface

//container

```
LinearLayout layout = new LinearLayout(this);  
layout.setOrientation(LinearLayout.VERTICAL);
```

//static element

```
TextView tv = new TextView(this);  
tv.setText("Title:");
```

//pixel conversion

```
DisplayMetrics displayMetrics = getResources().getDisplayMetrics();  
int hPixeli = (int) TypedValue.applyDimension(TypedValue.COMPLEX_UNIT_DIP, 20, displayMetrics);
```

Procedural interface

//height 20dp, width WRAP_CONTENT

```
LayoutParams lpt = new LayoutParams(LayoutParams.WRAP_CONTENT, hPixeli);  
tv.setLayoutParams(lpt);
```

//WRAP_CONTENT for editText and button

```
LayoutParams lp = new LayoutParams(LayoutParams.WRAP_CONTENT,  
LayoutParams.WRAP_CONTENT);
```

Procedural interface

//adding controls to the container

```
layout.addView(tv);  
layout.addView(et);  
layout.addView(button);
```

//displaying the container

```
setContentView(layout);
```

//editText

```
EditText et = new EditText(this);  
et.setText("Input the title");  
et.setEms(10);  
et.setLayoutParams(lp);
```

//button

```
Button buton = new Button(this);  
buton.setText("Send");  
buton.setLayoutParams(lp);
```

Container's parameters

Base class **ViewGroup.LayoutParams**

Defined in each container class

- `LinearLayout.LayoutParams`
- `RelativeLayout.LayoutParams`

Referred from a view

- `setLayoutParams()`
- `getLayoutParams()`

Common properties

- width, height

Container specific properties

- gravity, weight

Container's parameters

Constructor

- width, height

Example

//height 20 pixels

```
LinearLayout.LayoutParams lp = new LinearLayout.LayoutParams(LayoutParams.WRAP_CONTENT,  
20);
```

```
lp.setMargins(10, 10, 10, 10);//left, top, right, bottom (pixels)
```

```
view.setLayoutParams(lp);
```

```
LayoutParams lpb = view.getLayoutParams();
```

```
lpb.width = ViewGroup.LayoutParams.MATCH_PARENT;
```

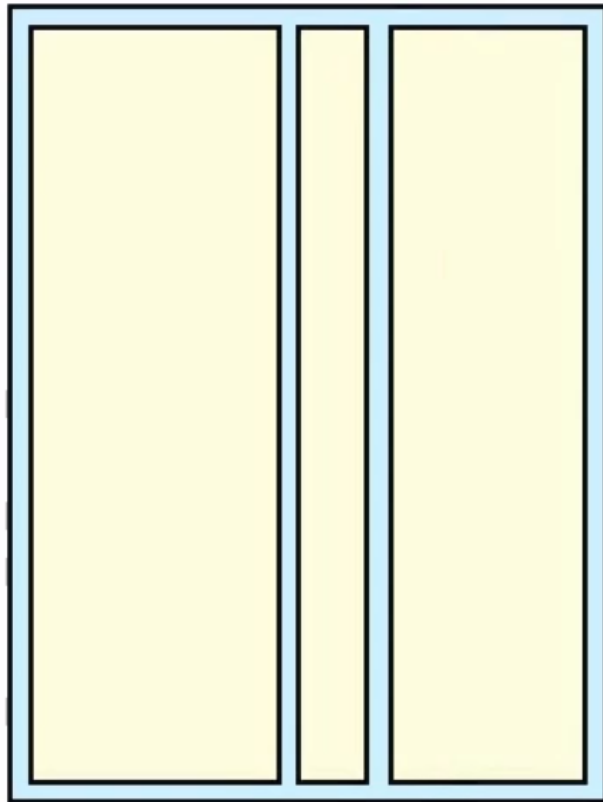
```
lpb.height = ViewGroup.LayoutParams.WRAP_CONTENT;
```

```
view.setLayoutParams(lpb);
```

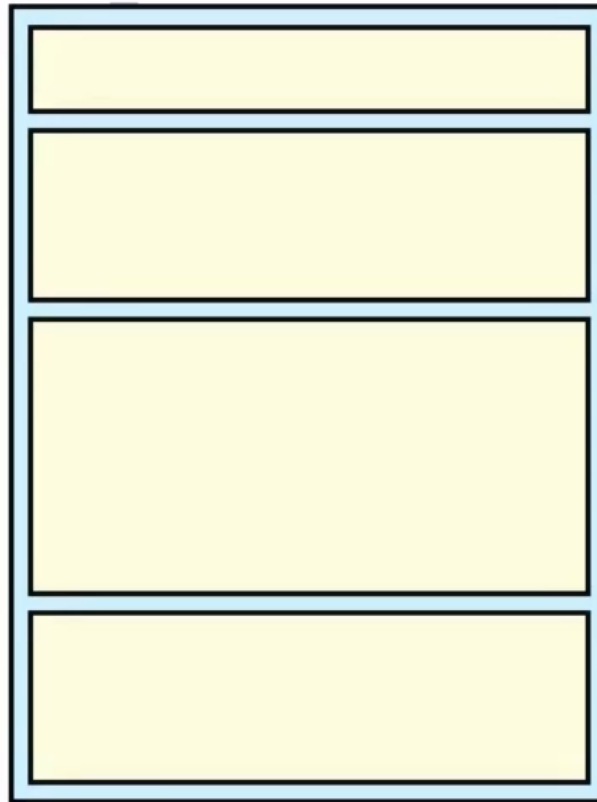
VIEW_GROUPS

CONTAINERS

horizontal



vertical



Linear layout

A layout that organizes its children into a single horizontal or vertical row. It creates a scrollbar if the length of the window exceeds the length of the screen.

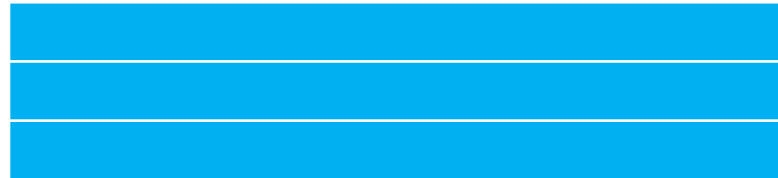
LinearLayout

android:orientation

- horizontal



- vertical



LinearLayout

Distributing remaining space (proportions)

- **android:layout_weight**

view1, view2, view3:
`android:layout_width="wrap_content"`



view1, view2, view3:
`android:layout_width="0dp"`
`android:layout_weight="1"`

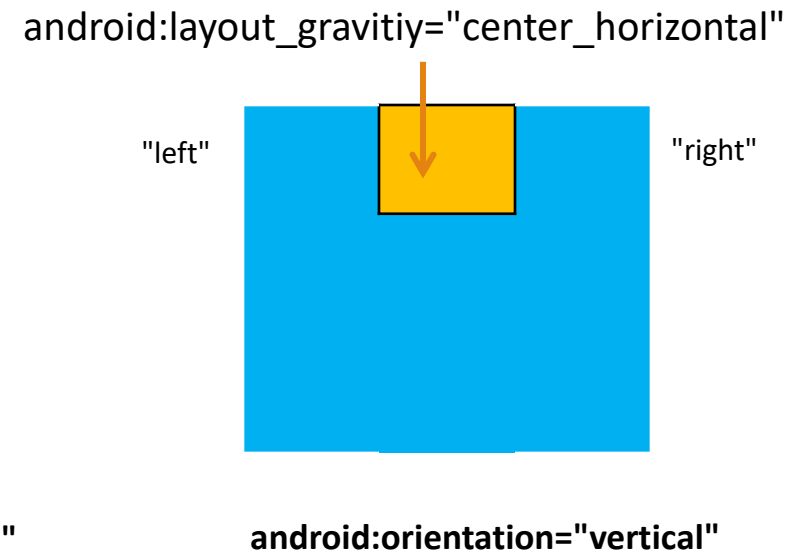
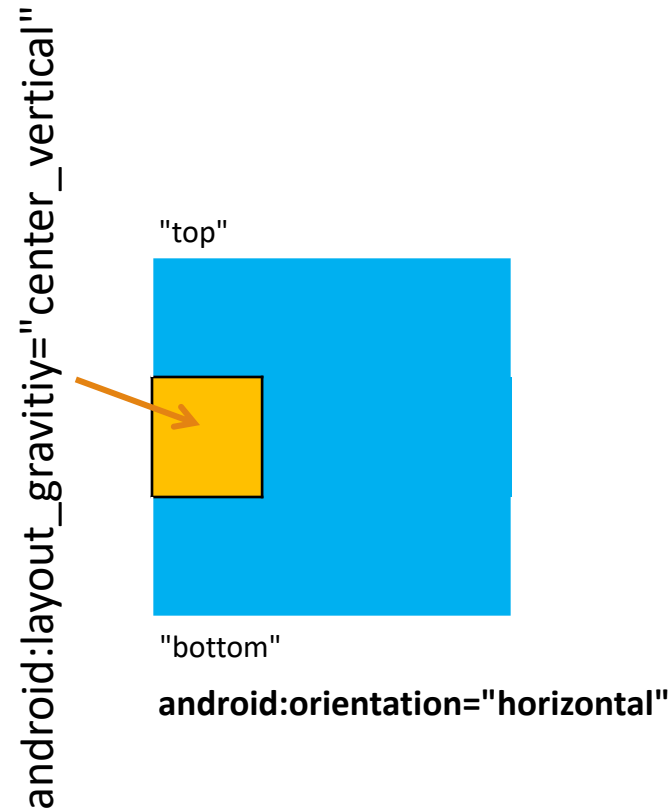


view1, view2
`android:layout_width="wrap_content"`
view3:
`android:layout_width="0dp"`
`android:layout_weight="1"`



LinearLayout

Control alignment inside the parent **android:layout_gravity**



Layout gravity vs Gravity



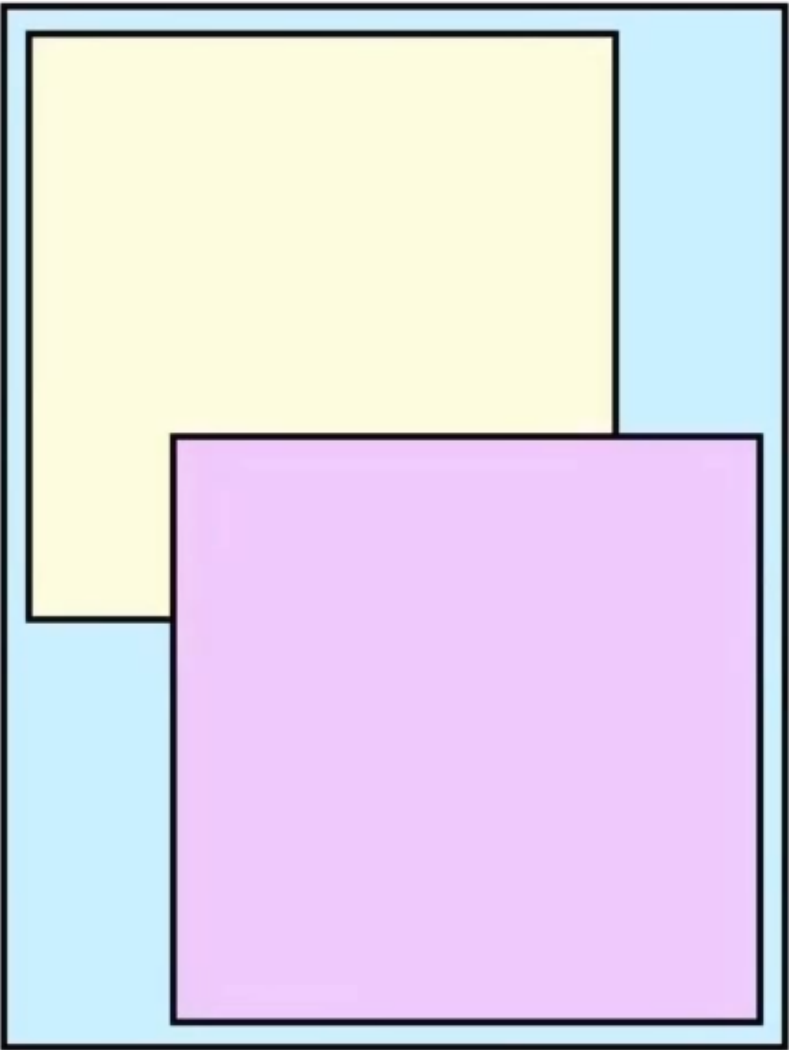
```
android:layout_gravity="right"  
android:gravity="center_horizontal|bottom"
```

FrameLayout

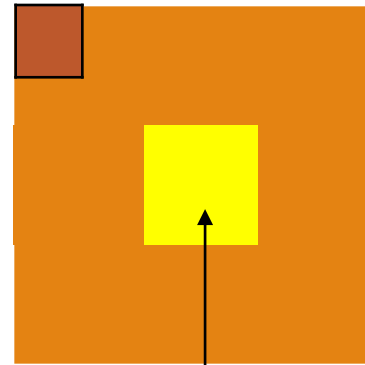
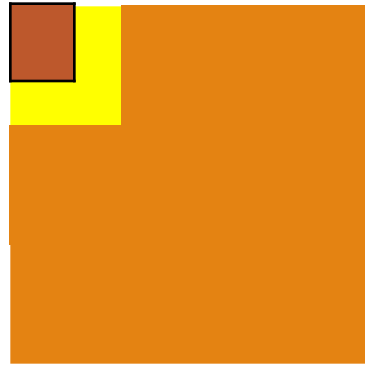
Usually used for managing one single visual element

Elements positioning:

- Implicitly: top-left corner
- Controls can overlap
- `android:layout_gravity`



FrameLayout



`android:layout_gravity="center"`

ScrollView/HorizontalScrollView

Can be used with a FrameLayout

Uses one single visual component

Uses scrollbars

- `android:scrollbars="vertical"`

Used for filling the entire available view

- `android:fillViewport="true"`

TableLayout

Derived from LinearLayout

Lines

- **TableRow**

Associated columns

- **android:layout_column**

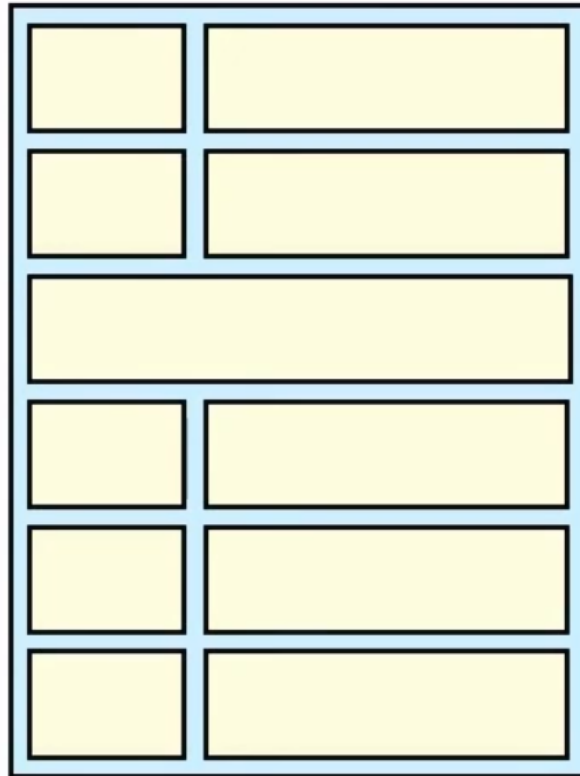
Spreading across multiple columns:

- **android:layout_span**

Content wrapping

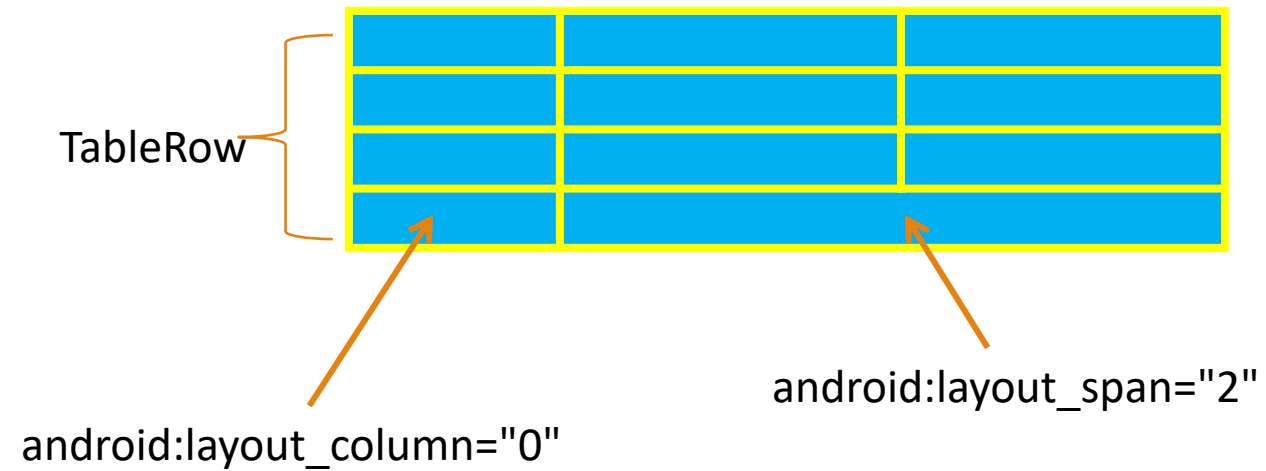
- **android:shrinkColumns**
 - remove free space
- **android:stretchColumns**
 - Extending columns to cover the remaining space
- **android:collapseColumns**
 - Hiding columns

TableLayout



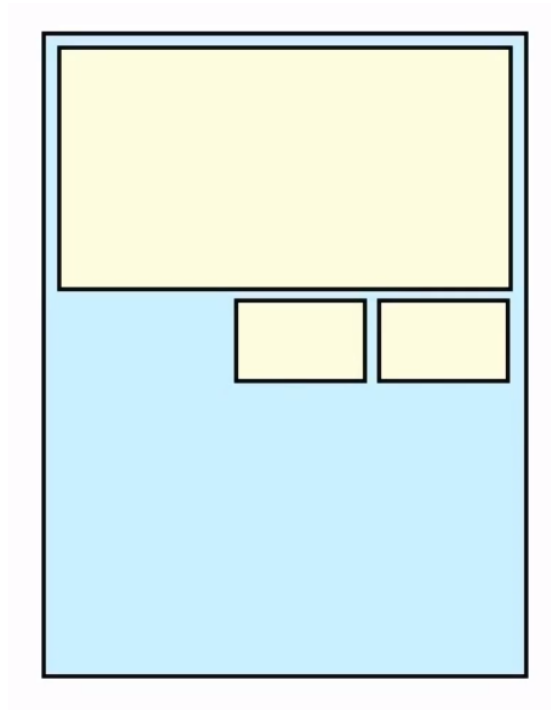
A diagram illustrating a TableLayout structure. It consists of a rectangular container divided into a grid of 7 rows and 2 columns. The cells are yellow with black borders. The layout is as follows:

TableLayout



Relative layout

Enables you to specify the location of child objects relative to each other (child A to the left of child B) or to the parent (aligned to the top of the parent).



RelativeLayout

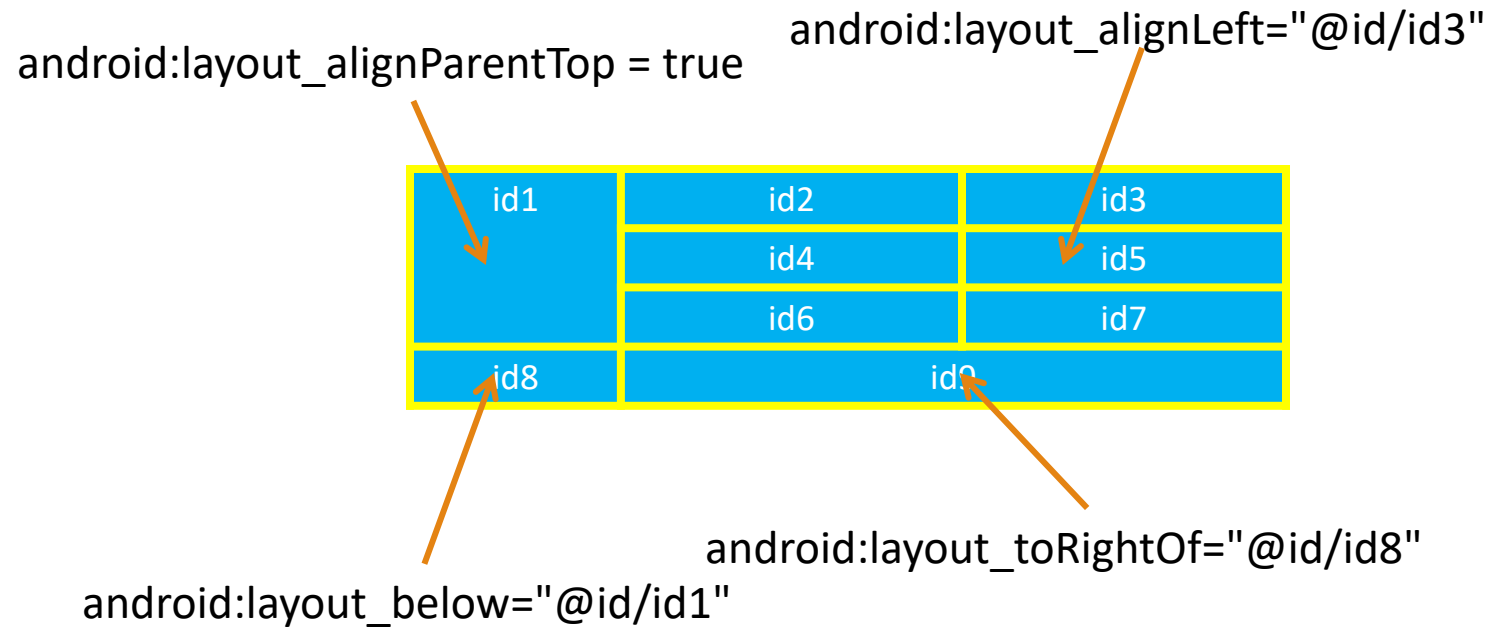
Alignment

- Relative to the container (true/false):
 - **android:layout_alignParent***Top | Left | Right | Bottom*
 - **android:layout_center***Horizontal | Vertical*
 - **android:layout_centerInParent**
- Relative to other components (id-ul):
 - **android:layout_align***Left | Right | Start | Top | End*

Positioning

- **android:layout_below**, **android:layout_above**
- **android:layout_to***End | Left | Start | Right***Of**

RelativeLayout



GridLayout

Number of columns

- **android:columnCount**

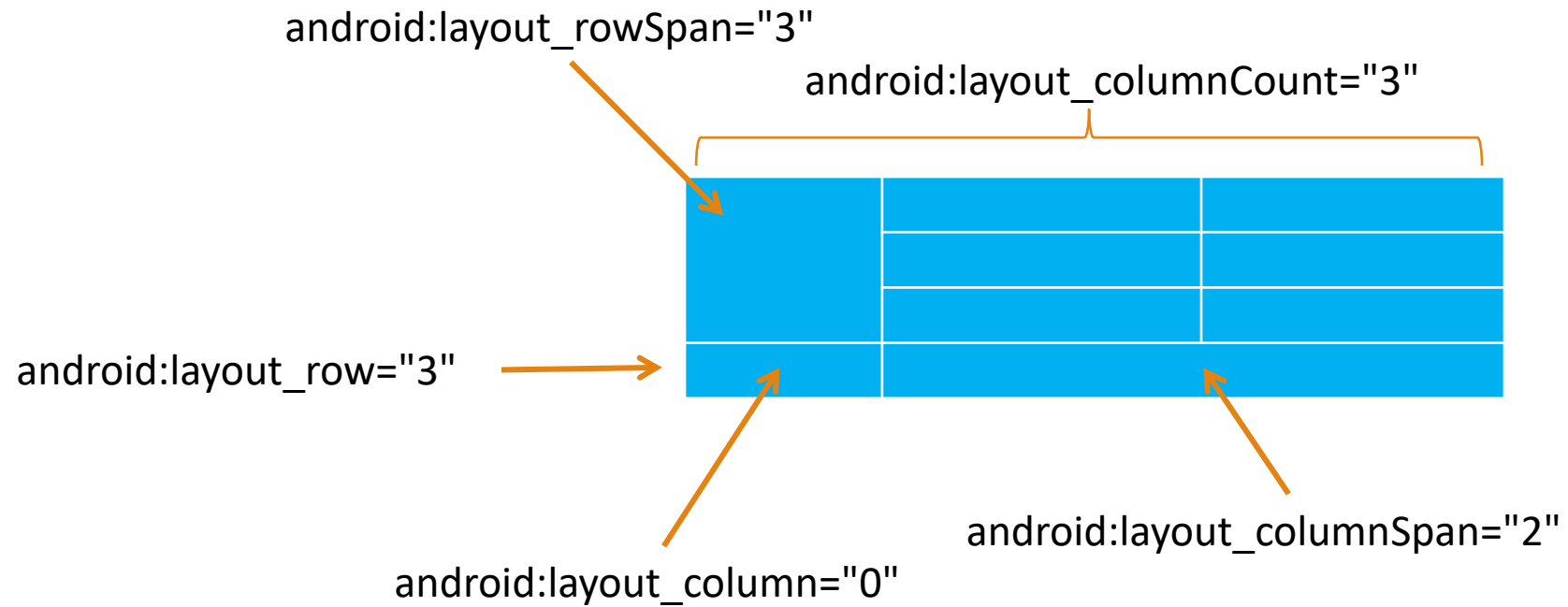
Spreading across multiple rows:

- **android:layout_rowSpan**

Spreading across multiple columns:

- **android:layout_columnSpan**

GridLayout



Include , Merge and ViewStub

Create separate layouts that can be included as elements into other layout (nesting) for easiness in control

Merge two or more elements in the same parent frame layout

Lazy loading one or more elements inside a view (at runtime), the system won't keep track of the unnecessary elements that are loaded onto the screen (deferred inflation)

Resources

C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9

S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012

R. Meier – Professional Android 4 Application Development, Wiley, 2012

P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012

P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015

<http://developer.android.com>